

Tutorial 10: Data Hazards (Solutions)

Solution: Q1 Data Hazards in a 5-Stage Pipeline

Given Instruction Sequence

I1: add \$t0, \$t1, \$t2

I2: lw \$t3, 0(\$t0)

I3: sub \$t4, \$t3, \$t5

I4: and \$t6, \$t4, \$t0

I5: sw \$t6, 0(\$t0)

Assumptions

A 5-stage pipeline (IF, ID, EX, MEM, WB) with split-cycle register access is used. Control hazards are ignored.

(a) Baseline Pipeline (No Forwarding)

RAW Hazards:

- I1 → I2 (t_0): EX→1st (stall)
- I2 → I3 (t_3): MEM→1st (stall)
- I3 → I4 (t_4): EX→1st (stall)
- I4 → I5 (t_6): EX→1st (stall)
- I1 → I4 (t_0): EX→3rd (no stall)
- I1 → I5 (t_0): EX→4th (no stall)

Pipeline Diagram:

Inst	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14
I1	IF	ID	EX	M	WB									
I2		IF	NOP	NOP	ID	EX	M	WB						
I3					IF	NOP	NOP	ID	EX	M	WB			
I4								IF	NOP	NOP	ID	EX	M	WB
I5											IF	NOP	NOP	ID

Stalls: Each close dependency requires 2 stall cycles.

Total stalls = 8

CPI:

$$\text{Total Cycles} = N + (k - 1) + \text{Stalls}$$

Where:

- N = Number of instructions
- k = Number of pipeline stages

$$\text{Total cycles} = 5 + (5 - 1) + 8 = 17$$

$$\text{CPI} = \frac{17}{5} = 3.4$$

(b) Effect of Split-Cycle

Split-cycle allows the **WB stage** of the producer to write to the register file in the first half of the cycle, while the **ID stage** of the consumer reads it in the second half of the same cycle. This effectively reduces the required "distance" between instructions by one full cycle.

(c) Pipeline with Forwarding

To minimize stalls, the following forwarding paths are utilized:

- **EX/MEM → EX (\$t0)**: Forwarding from I1 to I2.
- **MEM/WB → EX (\$t3)**: Forwarding from I2 to I3 (after a 1-cycle stall).
- **EX/MEM → EX (\$t4)**: Forwarding from I3 to I4.
- **EX/MEM → EX (\$t6)**: Forwarding from I4 to I5.

Remaining Hazard:

- **I2 → I3 (Load-Use)**: Because $t3$ is loaded from memory, it is not available until the end of Cycle 5 (MEM). I3 requires it at the start of its EX stage. Even with forwarding, a **1-cycle stall** is required to align the data availability with the EX stage.

Pipeline Diagram:

Inst	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
I1	IF	ID	EX	M	WB					
I2		IF	ID	EX	M	WB				
I3			IF	ID	st	EX	M	WB		
I4				IF	st	ID	EX	M	WB	
I5					st	IF	ID	EX	M	WB

Total stalls = 1

CPI:

$$\text{Total cycles} = 5 + 4 + 1 = 10$$

$$\text{CPI} = \frac{10}{5} = 2.0$$

(d) Hazard Detection Unit

The Hazard Detection Unit (HDU) detects load-use hazards and inserts a stall when required. If the current instruction is a load instruction and the next instruction requires the loaded value, the pipeline must be stalled.

Without HDU:

- The pipeline does not insert the required stall
- Forwarding alone cannot resolve the load-use hazard
- The dependent instruction uses an incorrect value
- This leads to incorrect program execution

(e) Performance Comparison

Solution:

Performance is evaluated using execution time:

$$\text{Execution Time} = \text{CPI} \times \text{Clock Cycle Time}$$

Clock Cycle Time:

The clock cycle time is determined by the slowest pipeline stage.

Without forwarding:

$$\text{Cycle time} = 200 \text{ ps}$$

With forwarding:

$$\text{EX stage} = 140 + 40 = 180 \text{ ps}$$

Since the maximum stage delay remains 200 ps (IF stage), the clock cycle time is unchanged.

Execution Time:

Without forwarding:

$$5 \times 3.4 \times 200 = 3400 \text{ ps}$$

With forwarding:

$$5 \times 2.0 \times 200 = 2000 \text{ ps}$$

Conclusion: The pipeline with forwarding is **1.7x faster**. Forwarding reduces CPI by minimizing stalls, thereby improving performance. Although forwarding increases the EX stage delay, it does not affect the overall clock cycle time in this case. In general, performance must be evaluated using execution time, as hardware optimizations may reduce CPI but can also increase cycle time.

Solution: Q2 Instruction Scheduling with Forwarding

Given Instruction Sequence

```
I1: lw $t0, 0($t1)
I2: add $t2, $t0, $t3
I3: lw $t4, 0($t5)
I4: add $t6, $t4, $t7
I5: add $t8, $t2, $t9
```

(a) Hazard Analysis

Question: Identify all RAW hazards and determine which cause stalls with forwarding.

Solution:

RAW Hazards:

- I1 → I2 (t_0): Load-use hazard $\Rightarrow$ 1 stall
- I2 → I3 (t_2): Resolved by forwarding (no stall)
- I3 → I4 (t_4): Load-use hazard $\Rightarrow$ 1 stall
- I2 → I5 (t_2): Resolved by forwarding (no stall)

Total Stalls = 2

Pipeline Diagram:

Cycle	1	2	3	4	5	6	7	8	9	10	11
I1: lw	IF	ID	EX	MEM	WB						
I2: add		IF	ID	NOP	EX	MEM	WB				
I3: lw			IF	NOP	ID	EX	MEM	WB			
I4: add					IF	ID	NOP	EX	MEM	WB	
I5: add						IF	NOP	ID	EX	MEM	WB

CPI:

$$\text{Total cycles} = 11, \quad \text{CPI} = \frac{11}{5} = 2.2$$

(b) Instruction Scheduling

Question: Reorder instructions to minimize stalls.

Solution:

By moving the independent instruction (I3) to the slot immediately following the first load (I1), we hide the latency.

```

I1: lw  $t0, 0($t1)
I3: lw  $t4, 0($t5)    (Independent work)
I2: add $t2, $t0, $t3   (Uses t0)
I4: add $t6, $t4, $t7   (Uses t4)
I5: add $t8, $t2, $t9   (Uses t2)

```

Explanation: The Load-Use latency of I1 is hidden by executing I3. Similarly, the Load-Use latency of I3 is hidden by executing I2. By the time I2 and I4 reach their EX stages, the data has already been fetched from memory and is available in the pipeline latches for forwarding.

(c) Performance Comparison

Question: Compute CPI before and after reordering.

Solution:

After Scheduling Pipeline Diagram:

Cycle	1	2	3	4	5	6	7	8	9
I1	IF	ID	EX	MEM	WB				
I3		IF	ID	EX	MEM	WB			
I2			IF	ID	EX	MEM	WB		
I4				IF	ID	EX	MEM	WB	
I5					IF	ID	EX	MEM	WB

CPI:

$$\text{Total cycles} = 9, \quad \text{CPI} = \frac{9}{5} = 1.8$$

Improvement: CPI reduces from 2.2 to 1.8.

(d) Conceptual Insight

1. **Why scheduling succeeds:** It succeeds because of **Instruction Level Parallelism (ILP)**. The code contains instructions that do not depend on each other, allowing the compiler to "fill" the structural wait-time of a load with useful work.

2. **Comparison with Failure Cases:** Scheduling fails when a code segment has a **strict serial dependency chain** (e.g., $I_1 \rightarrow I_2 \rightarrow I_3$). In such cases, no independent instructions exist to move into the stall slots, making the hardware stall unavoidable.

Q3. Effect of Functional Unit Design

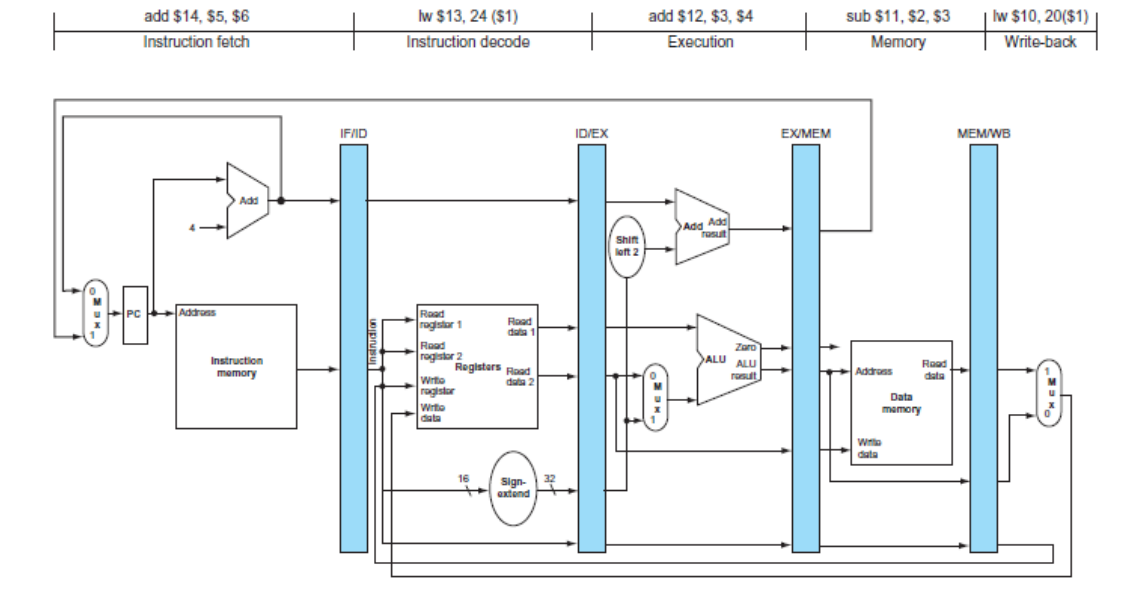


Figure 1: Pipelined Datapaths

In the pipelined datapaths shown in Figure 1. Assume that, among all the instructions executed by a processor, the following fractions given in Figure 2 exhibit a particular type of RAW data dependence. The type of RAW data dependence is identified by the stage that produces the result (EX or MEM) and the instruction that consumes the result (the 1st instruction that follows the one that produces the result, the 2nd instruction that follows, or both). We assume that the register write is done in the first half of the clock cycle and that register reads are done in the second half of the cycle, so “EX to 3rd” and “MEM to 3rd” dependences are not counted because they cannot result in data hazards. Also, assume the processor’s CPI is 1 if there are no data hazards.

EX to 1 st Only	MEM to 1 st Only	EX to 2 nd Only	MEM to 2 nd Only	EX to 1 st and MEM to 2 nd	Other RAW Dependences
5%	20%	5%	10%	10%	10%

Figure 2: Fraction of RAW data dependences

Assume the latencies given in Figure 3 for individual pipeline stages. For the EX stage,

latencies are provided separately for a processor without forwarding and for a processor with different forwarding mechanisms.

IF	ID	EX (no FW)	EX (full FW)	EX (FW from EX/MEM only)	EX (FW from MEM/ WB only)	MEM	WB
150 ps	100 ps	120 ps	150 ps	140 ps	130 ps	120 ps	100 ps

Figure 3: Latencies for each stage

1. If we use no forwarding, what fraction of cycles are we stalling due to data hazards?
2. If we use full forwarding (forward all results that can be forwarded), what fraction of cycles are we stalling due to data hazards? Which of the two forwarding options results in a shorter time per instruction?
3. Let us assume we cannot afford the three-input Mux units needed for full forwarding. We have to decide if it is better to forward only from the EX/MEM pipeline register (next-cycle forwarding) or only from the MEM/WB pipeline register (two-cycle forwarding). Which of the two options results in fewer data stall cycles?
4. For the given hazard probabilities and pipeline stage latencies, what is the speedup achieved by adding full forwarding to a pipeline that had no forwarding?

Solution

Given:

Dependence Type	Fraction
EX $\rightarrow$ 1st	5%
MEM $\rightarrow$ 1st	20%
EX $\rightarrow$ 2nd	5%
MEM $\rightarrow$ 2nd	10%
EX $\rightarrow$ 1st & MEM $\rightarrow$ 2nd	10%
Other	10%

IF	ID	EX (No FW)	EX (Full FW)	EX (EX/MEM)	EX (MEM/WB)	MEM	WB
150	100	120	150	140	130	120	100

(i) No Forwarding: Stall Fraction

Stall cycles:

Hazard	Stall Cycles
EX $\rightarrow$ 1st	2
MEM $\rightarrow$ 1st	2
EX $\rightarrow$ 2nd	1
MEM $\rightarrow$ 2nd	1
EX $\rightarrow$ 1st & MEM $\rightarrow$ 2nd	2

$$\text{Stall fraction} = 0.05(2) + 0.20(2) + 0.05(1) + 0.10(1) + 0.10(2)$$

$$= 0.10 + 0.40 + 0.05 + 0.10 + 0.20 = 0.85$$

Stall fraction = 0.85

(ii) Forwarding Analysis and Time per Instruction**(A) Full Forwarding**

Only load-use hazard remains:

$$\text{Stall fraction} = 0.20 \times 1 = 0.20$$

$$\text{CPI}_{FW} = 1 + 0.20 = 1.20$$

(B) Limited Forwarding Options**EX/MEM Forwarding Only**

$$\text{Stall fraction} = 0.75 \Rightarrow \text{CPI} = 1.75$$

MEM/WB Forwarding Only

$$\text{Stall fraction} = 0.65 \Rightarrow \text{CPI} = 1.65$$

(C) Time per Instruction

$$T = \text{CPI} \times \text{Clock Cycle Time}$$

Clock cycle time is determined by the slowest stage:

Cycle time = 150 ps

Full Forwarding

$$T_{FW} = 1.20 \times 150 = 180$$

EX/MEM Only

$$T = 1.75 \times 150 = 262.5$$

MEM/WB Only

$$T = 1.65 \times 150 = 247.5$$

MEM/WB forwarding performs better than EX/MEM

(iii) Best Limited Forwarding Option

$$\text{EX/MEM} = 0.75 \quad ; \quad \text{MEM/WB} = 0.65$$

MEM/WB forwarding gives fewer stalls

(iv) Speedup of Full Forwarding

$$\text{CPI}_{NF} = 1.85 \quad ; \quad \text{CPI}_{FW} = 1.20$$

$$\text{Speedup} = \frac{1.85}{1.20} \approx 1.54$$

Speedup ≈ 1.54

Final Summary

Scheme	CPI	Time	Performance
No Forwarding	1.85	High	Worst
Full Forwarding	1.20	180	Best
EX/MEM Only	1.75	262.5	Poor
MEM/WB Only	1.65	247.5	Better

Q4. Data Hazards and Pipeline Optimization

Consider a standard 5-stage pipelined processor:

IF → ID → EX → MEM → WB

The processor executes the following instruction sequence for 2x2 matrix addition:

```
# $r10 = base address of A
# $r11 = base address of B
# $r12 = base address of C
```

```
I1:  lw  $r1, 0($r10)      # A[0][0]
I2:  lw  $r2, 0($r11)      # B[0][0]
I3:  add $r3, $r1, $r2
I4:  sw  $r3, 0($r12)
```

```
I5:  lw  $r1, 4($r10)      # A[0][1]
I6:  lw  $r2, 4($r11)      # B[0][1]
I7:  add $r3, $r1, $r2
I8:  sw  $r3, 4($r12)
```

```
I9:  lw  $r1, 8($r10)      # A[1][0]
I10: lw  $r2, 8($r11)      # B[1][0]
I11: add $r3, $r1, $r2
I12: sw  $r3, 8($r12)
```

```
I13: lw  $r1, 12($r10)     # A[1][1]
I14: lw  $r2, 12($r11)     # B[1][1]
I15: add $r3, $r1, $r2
I16: sw  $r3, 12($r12)
```

To handle data hazards, different optimization schemes are proposed. The optimization schemes are briefly described as follows:

- **Hardware Optimisation:** EX/MEM→EX forwarding register is implemented.
- **Compiler Scheduling:** Instructions are rearranged to place independent instructions between dependent ones, avoiding stalls when possible.

The instruction mix contains 30% data-dependent instructions (RAW hazards), where:

- 70% are immediate dependencies

- 30% have one instruction gap

The stall cycles associated with each scheme are given below:

Scheme	Stall (Immediate Dependency)	Stall (1-gap Dependency)
Early Write-Back (Result in MEM stage)	1	0
Compiler Scheduling (80% success rate)	0 (if scheduled), else 2	0

Each scheme can be implemented independently.

- Draw the pipeline diagram for the program under each optimization scheme and illustrate the stall behavior.
- Analyze the pipeline behavior for the given instruction sequence under each scheme. Identify the stall cycles introduced.
- Compare the effectiveness of the schemes in reducing stalls for the given instruction sequence. Which scheme provides the best performance? Justify your answer based on pipeline timing.
- Using the given instruction mix, compute the effective CPI for each scheme.
- Determine the speedup of each scheme relative to the ideal pipeline ($CPI = 1$).

Solution

(i) Hardware Optimization (Forwarding)

Forwarding resolves ALU dependencies, but load-use hazards still introduce 1 stall.

Pipeline Diagram (First Dependency Block)

(i) Hardware Optimization (Forwarding)

Instruction	1	2	3	4	5	6	7	8	9
I1: lw	IF	ID	EX	MEM	WB				
I2: lw		IF	ID	EX	MEM	WB			
I3: add			IF	ID	ST	EX	MEM	WB	
I4: sw				IF	ST	ID	EX	MEM	WB

Observation: One stall cycle (ST) due to load-use hazard.

(ii) Compiler Scheduling**Scheduled Version (Ideal Case)**

```
lw r1
lw r2
lw next element    # filler instruction
add
```

Pipeline Diagram (No Stall Case)

Instruction	1	2	3	4	5	6	7
I1: lw	IF	ID	EX	MEM	WB		
I2: lw		IF	ID	EX	MEM	WB	
I5: lw			IF	ID	EX	MEM	WB
I3: add				IF	ID	EX	MEM

Observation: No stalls if scheduling succeeds.

Unsuccessful Scheduling Case

Instruction	1	2	3	4	5	6	7	8
I1: lw	IF	ID	EX	MEM	WB			
I2: lw		IF	ID	EX	MEM	WB		
I3: add			IF	ID	ST	ST	EX	MEM

Observation: 2 stall cycles when scheduling fails.

If scheduling successful: No stalls **If unsuccessful:** 2 stall cycles

0.1 Pipeline Behavior and Stall Analysis**Hardware Optimization**

Dependencies:

- lw → add (load-use hazard)
- add → sw (resolved via forwarding)

Each block introduces:

- 1 stall (lw $\rightarrow$ add)

Total blocks = 4

Total Stall Cycles = 4

Compiler Scheduling

- 80% hazards resolved $\rightarrow$ 0 stalls
- 20% unresolved $\rightarrow$ 2 stalls each

Expected stall per hazard:

$$= 0.8 \times 0 + 0.2 \times 2 = 0.4$$

0.2 Comparison of Schemes

Hardware Optimization

- Simple, predictable
- Always incurs 1 stall per load-use

Compiler Scheduling

- Potentially eliminates stalls
- Depends on instruction availability

Conclusion: Compiler scheduling gives better performance on average due to reduced stalls.

0.3 Effective CPI Calculation

Given:

- Data hazards = 30%
- Immediate = 70%
- 1-gap = 30%

(i) Hardware Optimization (Early Write-Back)

$$\begin{aligned}\text{CPI} &= 1 + (0.3 \times 0.7 \times 1) \\ &= 1 + 0.21 = 1.21\end{aligned}$$

(ii) Compiler Scheduling

$$\begin{aligned}\text{CPI} &= 1 + (0.3 \times 0.7 \times 0.4) \\ &= 1 + 0.084 = 1.084\end{aligned}$$

0.4 Speedup Calculation

$$\text{Speedup} = \frac{1}{\text{CPI}}$$

Hardware Optimization

$$= \frac{1}{1.21} \approx 0.83$$

Compiler Scheduling (Average)

$$= \frac{1}{1.084} \approx 0.92$$

Final Conclusion

- **Best performance:** Compiler Scheduling (Best Case, $\text{CPI} = 1$)
- **Most practical:** Compiler Scheduling (Average $\text{CPI} = 1.084$)
- **Most stable:** Hardware Optimization ($\text{CPI} = 1.21$)

Compiler scheduling provides the highest potential speedup but depends on instruction-level parallelism. Hardware optimization provides consistent but limited improvement.

5. Pipeline Hazards and Forwarding Analysis

Assume that the following instruction set is run in a standard 5-stage pipeline:

```
add $t0,$s0,$s1      # $t0 = $s0 + $s1
lw  $t1,4($t0)       # load using $t0
lw  $s0,0($s0)        # independent load
```

```

or  $t1,$t0,$t1    # uses $t1 and $t0
sw  $t1,0($t0)      # store result

```

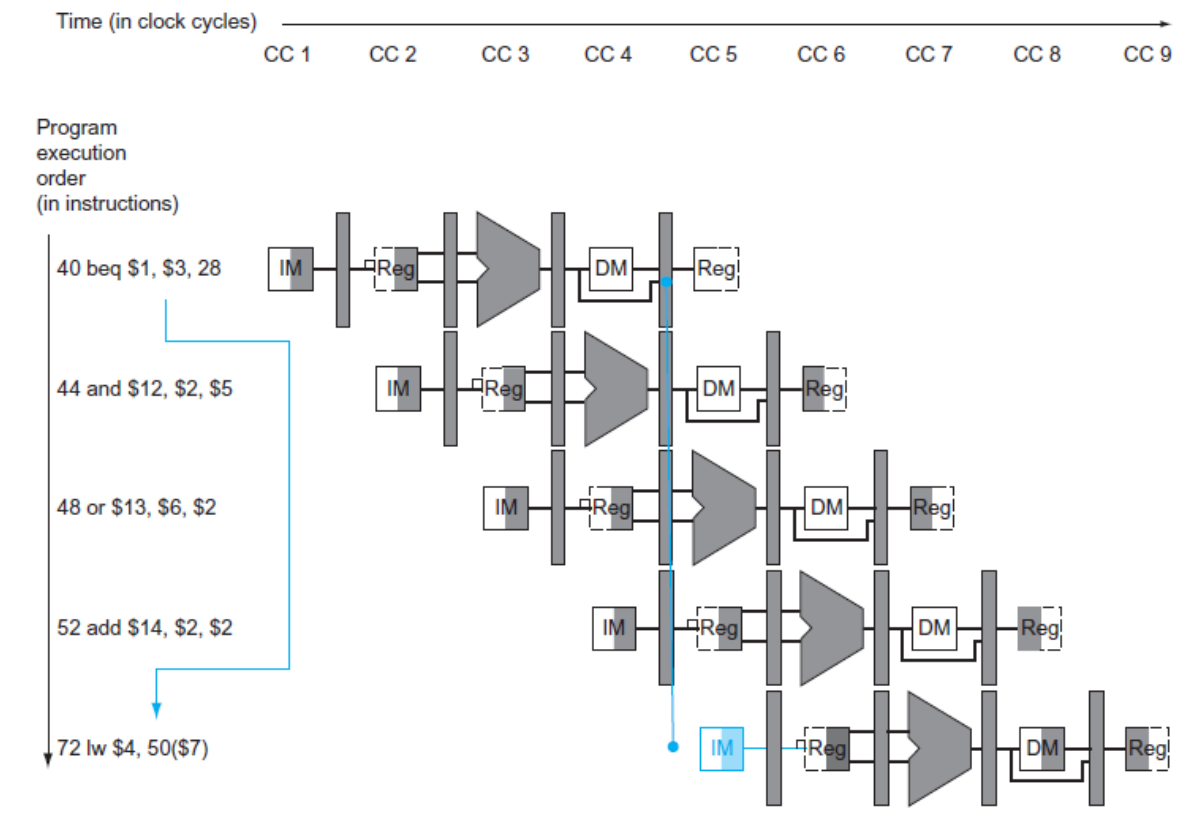


Figure 4: Pipelined Datapath with forwarding stages

Questions:

- If there is no forwarding or hazard detection, insert nops to ensure correct execution.
- Repeat (i) but now use nops only when a hazard cannot be avoided by rearranging instructions.
- If the processor has forwarding, but no hazard detection, what happens?
- With forwarding, specify hazard detection and forwarding signals for first five cycles.
- Without forwarding, what additional hazard detection signals are required?
- For (v), specify control signals for first five cycles.

(i) No Forwarding or Hazard Detection**Dependencies:**

- add → lw (via \$t0)

- `lw` $\rightarrow$ `or` (via `$t1`)
- `or` $\rightarrow$ `sw` (via `$t1`)

Since results are available only after WB stage, stalls must be inserted.

Correct Code:

```
add $t0,$s0,$s1
nop
nop
lw  $t1,4($t0)
lw  $s0,0($s0)
nop
nop
or  $t1,$t0,$t1
nop
nop
sw  $t1,0($t0)
```

(ii) Instruction Scheduling to Minimize NOPs

We move independent instructions to reduce stalls.

Optimized Code:

```
add $t0,$s0,$s1
lw  $s0,0($s0)      # independent instruction
lw  $t1,4($t0)
nop
or  $t1,$t0,$t1
nop
sw  $t1,0($t0)
```

Explanation:

- Independent load fills delay after add
- Some hazards still require NOPs

(iii) Forwarding but No Hazard Detection

Forwarding resolves ALU hazards but not load-use hazards.

Problematic Code Segment:

```
lw  $t1,4($t0)
or  $t1,$t0,$t1  # uses value too early
```

Result:

Incorrect execution due to use of stale data

Pipeline continues without stall, leading to incorrect result.

(iv) Forwarding + Hazard Detection (First 5 Cycles)

Instruction mapping:

- I1: add
- I2: lw t1
- I3: lw s0
- I4: or
- I5: sw

Cycle	Hazard Detection	Forwarding
1	None	None
2	None	None
3	None	EX/MEM → EX (add → lw)
4	None	MEM/WB → EX
5	Stall (load-use hazard)	Forwarding attempted

Explanation:

- Forwarding handles ALU dependencies
- Load-use hazard still requires stall

(v) Hazard Detection Unit Without Forwarding

Inputs Required:

- ID stage source registers (\$rs, \$rt)
- EX/MEM and MEM/WB destination registers
- RegWrite control signals

Outputs Required:

- PCWrite

- IF/IDWrite
- Control signals (to insert NOP)

Reason:

- To detect RAW hazards between instructions
- To stall pipeline since forwarding is absent

(vi) Hazard Signals for First 5 Cycles (No Forwarding)

Cycle	PCWrite	IF/IDWrite	Control (NOP)
1	1	1	0
2	1	1	0
3	0	0	1
4	0	0	1
5	1	1	0

Explanation:

- Cycles 3–4: Stall due to RAW hazard on \$t0
- Control signal inserts NOP

Conclusion

- Proper MIPS register usage improves clarity and correctness
- Without forwarding: significant stalls
- Scheduling reduces but cannot eliminate all hazards
- Forwarding + hazard detection is essential for correctness and performance

Q6. Hardware Support for Data Hazard Resolution

Consider a standard 5-stage pipeline as shown in Figure 1:

(a) What information must be stored in:

- ID/EX
- EX/MEM
- MEM/WB

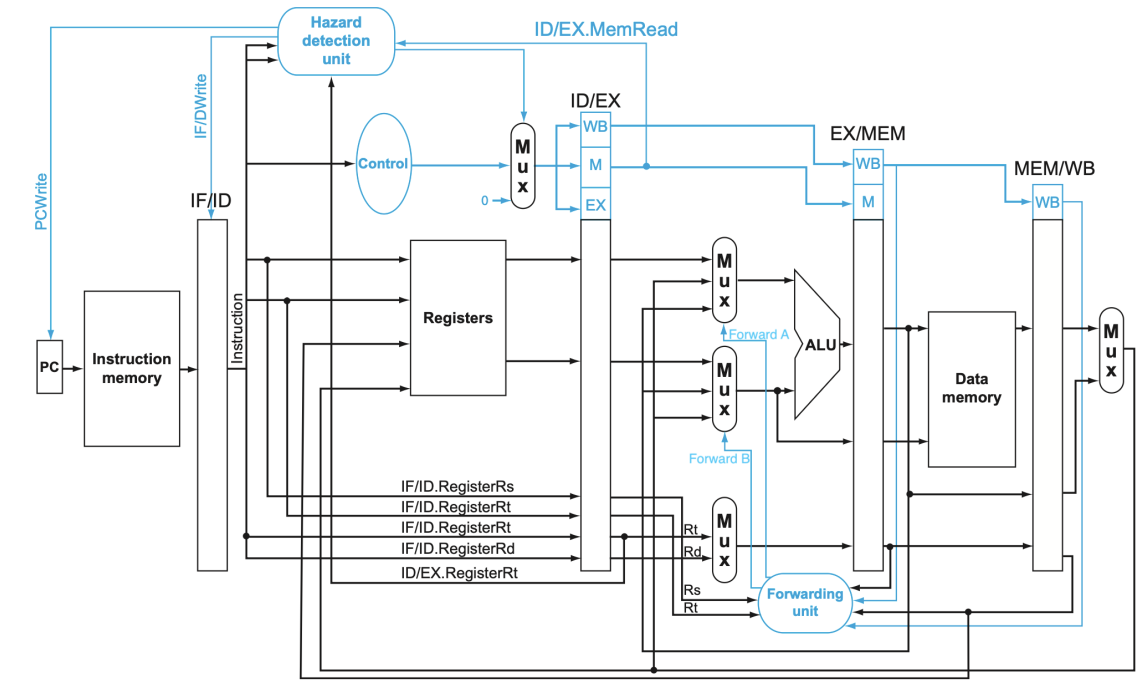


Figure 5:

(b) Assume that ForwardA and ForwardB are 2-bit control signals for the ALU input multiplexers with the following encoding:

- 00 $\rightarrow$ ID/EX
- 10 $\rightarrow$ EX/MEM
- 01 $\rightarrow$ MEM/WB

Design the forwarding conditions for ForwardA and ForwardB.

- (c) Explain how the forwarding unit resolves cases where multiple forwarding sources match.
- (d) Specify the precise condition (in terms of pipeline register fields) for detecting a load-use hazard.
- (e) What action should the hazard detection unit take when such a hazard is detected?
- (f) If forwarding is removed, what additional mechanisms or changes are required in the pipeline to ensure correct execution? Explain briefly.

Solution:

(a) Information Stored in Pipeline Registers

ID/EX

Data:

- ReadData1, ReadData2
- Rs, Rt, Rd
- Immediate value

Control Signals:

- EX: ALUOp, ALUSrc
- MEM: MemRead, MemWrite
- WB: RegWrite, MemtoReg

EX/MEM

Data:

- ALUResult
- WriteData (for store)
- Destination Register (Rd)

Control Signals:

- MEM: MemRead, MemWrite
- WB: RegWrite, MemtoReg

MEM/WB

Data:

- ReadData (from memory)
- ALUResult
- Destination Register

Control Signals:

- WB: RegWrite, MemtoReg

(b) Forwarding Conditions

ForwardA

If $EX/MEM.RegWrite = 1$ and $EX/MEM.Rd \neq 0$
 and $EX/MEM.Rd = ID/EX.Rs$
 $\Rightarrow ForwardA = 10$

Else if $MEM/WB.RegWrite = 1$ and $MEM/WB.Rd \neq 0$
 and $MEM/WB.Rd = ID/EX.Rs$
 $\Rightarrow ForwardA = 01$

Else $ForwardA = 00$

ForwardB

If $EX/MEM.RegWrite = 1$ and $EX/MEM.Rd \neq 0$
 and $EX/MEM.Rd = ID/EX.Rt$
 $\Rightarrow ForwardB = 10$

Else if $MEM/WB.RegWrite = 1$ and $MEM/WB.Rd \neq 0$
 and $MEM/WB.Rd = ID/EX.Rt$
 $\Rightarrow ForwardB = 01$

Else $ForwardB = 00$

(c) Resolving Multiple Matches

If both EX/MEM and MEM/WB match, priority is given to EX/MEM.

Reason:

- EX/MEM contains the most recent value
- MEM/WB contains an older value

Thus, EX/MEM is checked before MEM/WB.

(d) Load-Use Hazard Detection

If $ID/EX.MemRead = 1$
 and $((ID/EX.Rt = IF/ID.Rs) \vee (ID/EX.Rt = IF/ID.Rt))$

Explanation:

- Load result is available only after MEM stage
- Next instruction needs it in EX stage
- Forwarding cannot resolve this hazard

(e) Action on Hazard Detection

When hazard is detected:

- Stall PC: $PCWrite = 0$
- Stall IF/ID: $IF/IDWrite = 0$
- Insert bubble:
 - Set control signals in ID/EX to 0

Effect: One cycle stall is introduced.

(f) Without Forwarding

Problem

RAW hazards cannot be resolved early.

Solution

- Insert stalls until result reaches WB stage
- Hazard detection must check dependencies across pipeline stages

Example

ADD R1, R2, R3

SUB R4, R1, R5

SUB must wait until ADD completes WB stage.

Impact

- Increased CPI
- Reduced performance
- Simpler hardware